Obsah

ML Pipeline — WebAssembly Aplikácia **3**
Technická dokumentácia . . . 3
Obsah . . . 4
I. Teoretická časť . . . 4
II. Implementačná časť . . . 4
1. Úvod a motivácia . . . 4
Kľúčové funkcie . . . 4
Technologický stack . . . 5
2. Rust a WebAssembly — teoretické východiská . . . 5
2.1 WebAssembly (WASM) . . . 5
2.2 Jazyk Rust . . . 6
2.3 wasm-bindgen — most medzi Rust a JavaScript . . . 8
3. Teória strojového učenia . . . 8
3.1 Supervised Learning (Učenie s učiteľom) . . . 8
3.2 Lineárne modely . . . 9
3.3 K-Nearest Neighbors (KNN) . . . 10
3.4 Rozhodovacie stromy (Decision Trees) . . . 10
3.5 Preprocessing — teória . . . 11
4. Teória selekcie príznakov . . . 11
4.1 Problém dimenzionality (Curse of Dimensionality) . . . 11
4.2 Motivácia pre selekciu príznakov . . . 12
4.3 Taxonómia metód selekcie . . . 12
4.4 Pearsonova korelácia . . . 13
4.5 Chi-Square Test nezávislosti . . . 13
5. Metaheuristické optimalizačné metódy . . . 13
5.1 Úvod do metaheuristík . . . 13
5.2 Variable Neighborhood Search (VNS) . . . 13
5.3 Simulated Annealing (SA) . . . 14
5.4 Porovnanie VNS a SA . . . 15
5.5 Fitness funkcia v projekte . . . 16
6. Teória informácie a vzájomná informácia . . . 16
6.1 Shannonova entropia . . . 16
6.2 Podmienená entropia . . . 17
6.3 Vzájomná informácia (Mutual Information) . . . 17
6.4 Odhad MI pre spojité premenné — KSG algoritmus . . . 17
6.5 Information Gain . . . 18
7. Testovanie webových aplikácií — Playwright . . . 18
7.1 Problematika testovania webových aplikácií . . . 18
7.2 Úrovne testovania . . . 19
7.3 Prečo Playwright? . . . 19
7.4 Implementácia testov v projekte . . . 20
7.5 Pokrytie testov . . . 22
8. Architektúra systému . . . 22
Štruktúra projektu . . . 22
Vrstvy systému . . . 24
9. Návrhové vzory . . . 24

9.1 Builder Pattern — `MLPipelineBuilder` . . . 24
9.2 Factory Pattern . . . 25
9.3 Strategy Pattern . . . 25
9.4 Facade Pattern — `MLPipeline` . . . 25
9.5 Decorator Pattern — `ProcessorChain` . . . 26
9.6 Singleton Pattern — `CompatibilityRegistry` . . . 26
10. Modely strojového učenia . . . 26
10.1 Lineárna Regresia (`linreg`) . . . 26
10.2 Logistická Regresia (`logreg`) . . . 27
10.3 K-Nearest Neighbors (`knn`) . . . 27
10.4 Decision Tree (`tree`) . . . 28
11. Procesory dát (preprocessing) . . . 28
11.1 Numerické procesory . . . 28
11.2 Diskretizačné procesory . . . 29
11.3 Kódovacie procesory (pre kategorické dáta) . . . 30
11.4 Špeciálne procesory . . . 30
11.5 Reťazenie procesorov . . . 30
12. Selektory príznakov (feature selection) . . . 31
Prehľad selektorov . . . 31
12.1 Variance Threshold . . . 31
12.2 Correlation Selector . . . 32
12.3 Chi-Square Test . . . 32
12.4 Information Gain . . . 32
12.5 Mutual Information (KSG) . . . 32
12.6 SMC (Squared Multiple Correlation) . . . 32
12.7 Synergy VNS (Variable Neighborhood Search) . . . 33
12.8 Synergy SA (Simulated Annealing) . . . 34
13. Analýza cieľovej premennej (target analysis) . . . 35
13.1 Correlation Analyzer . . . 35
13.2 Mutual Information Analyzer . . . 36
13.3 SMC Analyzer (Squared Multiple Correlation) . . . 36
13.4 Entropy Analyzer . . . 36
Výstup analyzérov . . . 36
14. Embedded selektory . . . 36
14.1 Random Forest Selector . . . 36
14.2 Ridge Selector . . . 37
15. Mutual Information — KSG estimátor . . . 37
Algoritmus KSG (Kraskov-Stögbauer-Grassberger) . . . 37
16. Evaluácia modelov . . . 38
16.1 Klasifikačné metriky . . . 38
16.2 Regresné metriky . . . 39
17. Pipeline — životný cyklus . . . 39
17.1 Vytvorenie pipeline . . . 39
17.2 Trénovanie . . . 40
17.3 Deterministický split . . . 41
11.4 Cachovací systém . . . 41
18. WASM API rozhranie . . . 41
18.1 `WasmMLPipeline` — hlavný pipeline . . . 41

18.2 `WasmFactory` — factory pre frontend . . . 42
18.3 `WasmDataLoader` — načítavač dát . . . 42
19. Načítavanie dát . . . 42
Podporované formáty . . . 42
Proces načítania . . . 43
Ukážkový dataset . . . 43
20. Frontendová aplikácia . . . 44
Technológia . . . 44
Hlavné sekcie UI . . . 44
Modálne okná . . . 45
JavaScript architektúra . . . 45
Heatmapa . . . 45
21. Konfigurácia a nasadenie . . . 45
Požiadavky na build . . . 45
Závislosti (Cargo.toml) . . . 46
Buildovanie . . . 46
Build a Run skripty . . . 46
22. Automatizované testovanie — implementácia . . . 47
Framework . . . 47
Testové súbory a pokrytie . . . 47
Spustenie testov . . . 48
Testová infraštruktúra . . . 48
Postup tvorby testov . . . 49
23. Presety pipeline . . . 49
Klasifikačné presety . . . 49
Regresné presety . . . 49
Špeciálne . . . 50
24. Používateľská príručka . . . 50
Krok 1: Vytvorenie Pipeline . . . 50
Krok 2: Načítanie dát . . . 50
Krok 3: Analýza targetu (voliteľné) . . . 50
Krok 4: Selekcia príznakov . . . 51
Krok 5: Tréning a evaluácia . . . 51
Krok 6: Editor dát (voliteľné) . . . 51

ML Pipeline — WebAssembly Aplikácia

Technická dokumentácia

Verzia: 0.1.0
Technológie: Rust, WebAssembly, HTML/CSS/JS
Autor: Matúš
Dátum: Marec 2026

Obsah

I. Teoretická časť

1. Úvod a motivácia
2. Rust a WebAssembly — teoretické východiská
3. Teória strojového učenia
4. Teória selekcie príznakov
5. Metaheuristické optimalizačné metódy
6. Teória informácie a vzájomná informácia
7. Testovanie webových aplikácií — Playwright

II. Implementačná časť

8. Architektúra systému
9. Návrhové vzory
10. Modely strojového učenia
11. Procesory dát (preprocessing)
12. Selektory príznakov (feature selection)
13. Analýza cieľovej premennej (target analysis)
14. Embedded selektory
15. Mutual Information — KSG estimátor
16. Evaluácia modelov
17. Pipeline — životný cyklus
18. WASM API rozhranie
19. Načítavanie dát
20. Frontendová aplikácia
21. Konfigurácia a nasadenie
22. Automatizované testovanie — implementácia
23. Presety pipeline
24. Používateľská príručka

---

1. Úvod a motivácia

Táto aplikácia predstavuje **kompletný ML pipeline**, ktorý beží priamo v prehliadači vďaka technológii **WebAssembly (WASM)**. Celá výpočtová logika je napísaná v jazyku **Rust** a skompilovaná do WASM modulu, čo umožňuje:

- **Výkon blízky natívnemu** — Rust kód beží takmer rovnako rýchlo ako natívna aplikácia
- **Žiadny server** — všetky výpočty prebiehajú na strane klienta, nevyžaduje sa žiadny backend
- **Portabilita** — aplikácia funguje v akomkoľvek modernom prehliadači
- **Bezpečnosť dát** — dáta nikdy neopúšťajú zariadenie používateľa

Kľúčové funkcie

Funkcia	Popis
Trénovanie modelov	Logistická/lineárna regresia, KNN, Decision Tree
Preprocessing dát	15+ procesorov (scaling, encoding, transformácie)
Selekcia príznakov	8 metód vrátane metaheuristík (VNS, SA)
Analýza targetu	4 metódy analýzy cieľovej premennej
Vizualizácie	Heatmapa korelácií/MI, interaktívna mapa príznakov
Editor dát	In-browser editácia dátovej sady
Evaluácia	18+ metrík pre klasifikáciu a regresiu

Technologický stack

```
┌──────────────────────────────────────────────┐
│            Frontend (index.html)             │
│         HTML / CSS / JavaScript              │
│    Single-Page Application, 3600+ riadkov    │
├──────────────────────────────────────────────┤
│          WebAssembly Bridge                  │
│         wasm-bindgen + serde                 │
├──────────────────────────────────────────────┤
│           Rust Core Engine                   │
│   SmartCore ML · KD-tree · Statrs            │
│   ~5000 riadkov Rust kódu                    │
└──────────────────────────────────────────────┘
```

---

2. Rust a WebAssembly — teoretické východiská

2.1 WebAssembly (WASM)

WebAssembly (skrátene WASM) je binárny inštrukčný formát navrhnutý ako prenosný cieľ kompilácie pre vyššie programovacie jazyky. Bol štandardizovaný konzorciom W3C v roku 2019 a je natívne podporovaný vo všetkých hlavných prehliadačoch (Chrome, Firefox, Safari, Edge).

Prečo WebAssembly? Tradičný JavaScript je **interpretovaný jazyk** s JIT (Just-In-Time) kompiláciou. Hoci moderné JS enginy (V8, SpiderMonkey) dosahujú pôsobivý výkon, majú inherentné obmedzenia:

Aspekt	JavaScript	WebAssembly
Typový systém	Dynamický — typy sa určujú za behu	Statický — typy sú známe pri kompilácii
Pamäťový model	Garbage Collector — nepredvídateľné pauzy	Manuálna/RAII — deterministické uvoľňovanie
Numerické výpočty	IEEE 754 float64 pre všetky čísla	Natívne i32, i64, f32, f64
Optimalizácia	JIT — optimalizuje za behu	AOT — plne optimalizované pred spustením
Veľkosť binárky	Zdrojový kód (minifikovaný)	Kompaktný binárny formát
Predvídateľnosť	Deoptimalizácia pri type mismatch	Konzistentný výkon

Pre **výpočtovo náročné úlohy** ako trénovanie ML modelov, výpočet vzájomnej informácie, alebo maticové operácie, WASM dosahuje výkon **2-10× lepší** než ekvivalentný JavaScript kód.

Architektúra WASM modulu

```
┌──────────────────────────────────┐
│      JavaScript Runtime          │
│   (DOM, Event Loop, Fetch)       │
├──────────────────────────────────┤
│     WebAssembly Interface        │      ← wasm-bindgen generuje
│   (Import/Export funkcie)        │        "glue" kód automaticky
├──────────────────────────────────┤
│     WebAssembly Module           │
│  (Lineárna pamäť, Stack)         │     ← Kompilovaný Rust kód
│  ┌──────────────────────────┐    │
│  │   Linear Memory          │    │      ← Zdieľaná pamäť pre dáta
│  │   (ArrayBuffer)          │    │
│  └──────────────────────────┘    │
└──────────────────────────────────┘
```

Bezpečnostný model WASM beží v **sandboxovanom prostredí**: - **Izolovaná pamäť** — modul nemá prístup k pamäti hostiteľa - **Žiadny priamy prístup k DOM** — musí komunikovať cez JS bridge - **Validácia pred spustením** — binárka je validovaná pred execúciou - **Same-Origin Policy** — rovnaké bezpečnostné pravidlá ako pre JS

2.2 Jazyk Rust

Rust je systémový programovací jazyk od Mozilla Research (2010, stable 1.0 v 2015). Je ideálny pre WASM kompiláciu z viacerých dôvodov:

Bezpečnosť pamäte bez Garbage Collectora Rust využíva systém **ownership a borrowing**, ktorý zaručuje bezpečnosť pamäte v čase kompilácie:

```rust
// Ownership — každá hodnota má práve jedného vlastníka
let data = vec![1, 2, 3];    // data vlastní vektor
let reference = &data;         // nemeniteľná referencia (borrow)
// data nemôže byť modifikovaná kým existuje &data

// Lifetime — kompilátor sleduje platnosť referencií
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() { x } else { y }
}
```

Výhody oproti iným jazykom:

Jazyk	Pamäťový model	Problém pre WASM
C/C++	Manuálna správa	Use-after-free, buffer overflow, memory leaks
Java/C#	Garbage Collector	GC musí byť zabalený do WASM binárky (~2MB navyše)
Go	Garbage Collector + Runtime	Veľký runtime overhead
Rust	**Ownership + RAII**	**Žiadny GC, žiadny runtime overhead**

Zero-Cost Abstractions Rustové abstrakcie (generics, traits, iterators) sú **plne rozbalené pri kompilácii** — výsledný strojový kód je rovnako rýchly ako ručne napísaný kód v C:

```rust
// Tento kód sa skompiluje do rovnakého strojového kódu
// ako ručne napísaný cyklus
let sum: f64 = data.iter()
    .filter(|x| **x > 0.0)
    .map(|x| x * x)
    .sum();
```

Ekosystém pre WASM

Nástroj	Popis
`wasm-pack`	Build nástroj — kompilácia, optimalizácia, generovanie JS balíka
`wasm-bindgen`	Automatický bridge medzi Rust a JS typmi
`serde-wasm-bindgen`	Serializácia Rust štruktúr ↔ JS objektov (JsValue)
`web-sys`	Bindings na Web API (console.log, DOM, fetch)
`js-sys`	Bindings na JS built-in objekty (Array, Object, Promise)

2.3 wasm-bindgen — most medzi Rust a JavaScript

`wasm-bindgen` automaticky generuje „lepidlový" (glue) kód pre komunikáciu medzi Rust a JS:

```rust
use wasm_bindgen::prelude::*;

#[wasm_bindgen]
pub struct WasmMLPipeline {
    inner: MLPipeline,
}

#[wasm_bindgen]
impl WasmMLPipeline {
    #[wasm_bindgen(constructor)]
    pub fn new() -> Self { /* ... */ }

    pub fn loadData(&mut self, data: &str, target: &str) -> JsValue {
        // Rust typy sa automaticky konvertujú na JS typy
        serde_wasm_bindgen::to_value(&result).unwrap()
    }
}
```

Generovaný JS wrapper (`wasm.js`) potom vyzerá:

```javascript
import * as wasm from './wasm_bg.wasm';

export class WasmMLPipeline {
    constructor() { this.__wbg_ptr = wasm.wasmmlpipeline_new(); }
    loadData(data, target) { return wasm.wasmmlpipeline_loadData(this.__wbg_ptr, da
}
```

Výhody pre túto aplikáciu

- **Žiadny server** — všetky ML výpočty bežia na klientovi
- **Privátnosť dát** — dáta nikdy neopustia prehliadač
- **Offline schopnosť** — po načítaní funguje bez internetu
- **Distribuovaný výpočet** — každý klient má svoj vlastný „ML server"
- **Kompaktná binárka** — celý ML engine v ~400 KB WASM súbore

---

3. Teória strojového učenia

3.1 Supervised Learning (Učenie s učiteľom)

Strojové učenie s učiteľom je paradigma, kde model sa učí z **označených trénovacích dát** — párov vstupov X a požadovaných výstupov y. Cieľom je naučiť sa mapovanie $f : X \to y$ tak, aby model dokázal **generalizovať** na nové, nevidené dáta.

Formálna definícia Máme trénovací dataset $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$, kde: - $x_i \in \mathbb{R}^d$ — vektor príznakov (features) s d dimenziami - $y_i \in \mathcal{Y}$ — cieľová premenná (target)

Pre **regresiu**: $\mathcal{Y} = \mathbb{R}$ (spojité hodnoty)
Pre **klasifikáciu**: $\mathcal{Y} = \{0, 1, \dots, K-1\}$ (diskrétne triedy)

Bias-Variance Tradeoff Celková chyba modelu sa rozkladá na:

$$E[(y - \hat{f}(x))^2] = \underbrace{\text{Bias}^2[\hat{f}(x)]}_{\text{podhodnotenie}} + \underbrace{\text{Var}[\hat{f}(x)]}_{\text{preučenie}} + \underbrace{\sigma^2}_{\text{šum}}$$

- **Bias** — systematická chyba modelu (príliš jednoduchý model)
- **Variance** — citlivosť na trénovacie dáta (príliš zložitý model)
- **Irreducible error** — šum v dátach, nedá sa odstrániť

3.2 Lineárne modely

Lineárna regresia Predpokladá lineárny vzťah medzi príznakmi a targetom:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_d x_d = X\beta$$

Parametre β sa nájdu **minimalizáciou MSE** (Mean Squared Error):

$$\beta^* = \arg\min_{\beta} \sum_{i=1}^{n} (y_i - x_i^T \beta)^2$$

Riešenie v uzavretom tvare (Normal Equation):

$$\beta^* = (X^T X)^{-1} X^T y$$

Implementácia v projekte podporuje dva riešiče: - **QR dekompozícia** — numericky stabilnejšia, preferovaná - **SVD dekompozícia** — robustnejšia pre singulárne matice

Logistická regresia Pre binárnu klasifikáciu modeluje **pravdepodobnosť príslušnosti k triede** pomocou sigmoid funkcie:

$$P(y = 1|x) = \sigma(x^T \beta) = \frac{1}{1 + e^{-x^T \beta}}$$

Optimalizácia maximalizuje **log-likelihood** s L2 regularizáciou:

$$\mathcal{L}(\beta) = \sum_{i=1}^{n} [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)] - \frac{\alpha}{2} \|\beta\|^2$$

Parameter α kontroluje silu regularizácie — väčšia hodnota penalizuje veľké koeficienty a redukuje overfitting.

3.3 K-Nearest Neighbors (KNN)

KNN je **neparametrický** algoritmus — neučí sa explicitné parametre, ale ukladá celý trénovací dataset. Pre novú vzorku x:

1. Nájdi k najbližších susedov v trénovacej sade
2. Pre regresiu: $\hat{y} = \frac{1}{k}\sum_{i \in N_k(x)} y_i$ (priemer)
3. Pre klasifikáciu: $\hat{y} = \text{mode}_{i \in N_k(x)} y_i$ (väčšinové hlasovanie)

Euklidovská vzdialenosť:

$$d(x, x') = \sqrt{\sum_{j=1}^{d} (x_j - x'_j)^2}$$

Voľba k je kritická: - **Malé k** (napr. 1) — vysoká variance, citlivé na šum - **Veľké k** (napr. 50) — vysoký bias, nadmerné vyhladzovanie - **Optimálne k** — typicky $k \approx \sqrt{n}$, overené krížovou validáciou

3.4 Rozhodovacie stromy (Decision Trees)

Rozhodovacie stromy rekurzívne delia priestor príznakov na podoblasti, pričom v každej čím sa maximalizuje **informačná čistota** výsledných uzlov.

Kritérium rozdelenia Pre každý uzol sa hľadá najlepší príznak j a prahová hodnota t:

$$\text{Split}^* = \arg\max_{j,t} \left[\text{Impurity}(\text{parent}) - \sum_{child} \frac{n_{child}}{n_{parent}} \text{Impurity}(child) \right]$$

Bežné miery nečistoty: - **Gini impurity**: $G = 1 - \sum_k p_k^2$ - **Entropia**: $H = -\sum_k p_k \log_2 p_k$ - **MSE** (pre regresiu): $\frac{1}{n}\sum(y_i - \bar{y})^2$

Regularizácia stromu

Parameter	Efekt
`max_depth`	Obmedzuje hĺbku stromu — redukuje overfitting
`min_samples_split`	Minimálny počet vzoriek pre ďalšie delenie

3.5 Preprocessing — teória

Preprocessing je **nevyhnutný krok** v ML pipeline, pretože:

1. Numerická stabilita

Algoritmy založené na vzdialenosti (KNN) a gradientové metódy (logistická regresia) sú citlivé na **škálu príznakov**. Bez normalizácie by príznak s rozsahom [0, 100000] dominoval nad príznakom s rozsahom [0, 1].

2. Kódovanie kategorických dát

ML modely pracujú s číselnými vektormi. Kategorické dáta (napr. „muž"/„žena") musia byť konvertované na čísla. Existuje viacero prístupov:

Metóda	Princíp	Vhodné pre
Label Encoding	Priradí celé číslo	Ordinálne dáta
One-Hot Encoding	Binárny vektor	Nominálne dáta (bez poradia)
Target Encoding	Priemer targetu	Vysoká kardinalita
Frequency Encoding	Frekvencia výskytu	Vysoká kardinalita

3. Transformácia rozdelenia

Mnohé modely predpokladajú **normálne rozdelenie** príznakov. Skosené (skewed) rozdelenia zhoršujú výkon: - **Log transformácia**: $x' = \ln(x + c)$ — pre pravostranné skosy - **Box-Cox**: $x' = \frac{x^\lambda - 1}{\lambda}$ — generalizovaná transformácia - **Yeo-Johnson**: Rozšírenie Box-Cox na negatívne hodnoty

4. Ošetrenie odľahlých hodnôt

Odľahlé hodnoty (outliers) výrazne ovplyvňujú modely citlivé na extrémy (lineárna regresia, KNN): - **IQR metóda**: hodnoty mimo $[Q_1 - 1.5 \cdot IQR, Q_3 + 1.5 \cdot IQR]$ - **Z-score**: hodnoty s $|z| > 3$ sa považujú za outliers

---

4. Teória selekcie príznakov

4.1 Problém dimenzionality (Curse of Dimensionality)

Richard Bellman (1961) opísal fenomén zvaný **kliatba dimenzionality**. S rastúcim počtom príznakov d:

- Objem priestoru **exponenciálne rastie** → dáta sa stávajú riedkymi
- Na pokrytie priestoru potrebujeme **exponenciálne viac dát**: $n \sim O(c^d)$
- Vzdialenosť medzi bodmi sa stáva **takmer konštantnou** (všetky body sú rovnako ďaleko)
- Modely trpia **overfittingom** — učia sa šum namiesto signálu

4.2 Motivácia pre selekciu príznakov

Feature selection rieši tieto problémy výberom **podmnožiny relevantných príznakov** $S \subseteq \{1, 2, \dots, d\}$:

Výhoda	Vysvetlenie
Redukcia overfittingu	Menej parametrov = jednoduchší model
Zrýchlenie tréningu	Menej príznakov = rýchlejšie matice
Interpreovateľnosť	Transparentnosť — ktoré príznaky sú dôležité
Odstránenie šumu	Irelevantné features vnášajú len šum

4.3 Taxonómia metód selekcie

```
Metódy selekcie príznakov
├── Filter metódy
│   ├── Univariátne (každý príznak nezávisle)
│   │   ├── Variance Threshold
│   │   ├── Pearsonova korelácia
│   │   ├── Chi-Square test
│   │   └── Mutual Information
│   └── Multivariátne (vzťahy medzi príznakmi)
│       ├── SMC (Squared Multiple Correlation)
│       └── mRMR (Minimum Redundancy Maximum Relevance)
├── Wrapper metódy
│   ├── Forward Selection
│   ├── Backward Elimination
│   └── Metaheuristiky
│       ├── VNS (Variable Neighborhood Search)
│       └── SA (Simulated Annealing)
└── Embedded metódy
    ├── L1 regularizácia (Lasso)
    ├── L2 regularizácia (Ridge)
    └── Feature importance stromov
```

Filter metódy Hodnotia príznaky **nezávisle od modelu** — rýchle, ale nezvažujú interakcie:

$$\text{Score}(f_i) = g(f_i, y)$$

kde g je metrika (korelácia, MI, χ^2, variancia).

Wrapper metódy Hodnotia **podmnožiny príznakov** na základe výkonu konkrétneho modelu — presnejšie, ale výpočtovo náročné:

$$\text{Quality}(S) = \text{ModelPerformance}(f_S, y)$$

Prehľadávaný priestor má 2^d podmnožín → exhaustívne prehľadávanie je NP-ťažké → potrebujeme **metaheuristiky**.

Embedded metódy Selekcia je **súčasťou procesu tréningu** — model priamo určuje dôležitosť: - **Ridge (L2)**: $\|\beta\|_2^2$ → malé koeficienty = nedôležité features - **Random Forest**: Pokles impurity pri rozdelení = dôležitosť

4.4 Pearsonova korelácia

Meria **lineárny** vzťah medzi dvoma premennými:

$$r_{XY} = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2} \cdot \sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}}$$

kde $r \in [-1, 1]$: - $|r| = 1$ — perfektná lineárna závislosť - $r = 0$ — žiadna lineárna závislosť (môže existovať nelineárna!)

4.5 Chi-Square Test nezávislosti

Test hypotézy H_0: príznak X a target Y sú nezávislé:

$$\chi^2 = \sum_{i=1}^{r}\sum_{j=1}^{c}\frac{(O_{ij} - E_{ij})^2}{E_{ij}}$$

kde: - O_{ij} — pozorovaná frekvencia v bunke (i, j) kontigenčnej tabuľky - $E_{ij} = \frac{R_i \cdot C_j}{N}$ — očakávaná frekvencia za predpokladu nezávislosti

Vysoká χ² hodnota → zamietame H_0 → príznak je **závislý** od targetu → užitočný.

---

5. Metaheuristické optimalizačné metódy

5.1 Úvod do metaheuristík

Metaheuristiky sú **aproximačné optimalizačné algoritmy** navrhnuté pre riešenie NP-ťažkých problémov, kde presné metódy zlyhávajú kvôli exponenciálnej zložitosti. Pre selekciu príznakov je prehľadávaný priestor 2^d podmnožín — pri 20 príznakoch to je vyše 1 milión kombinácií.

5.2 Variable Neighborhood Search (VNS)

Navrhli Mladenović a Hansen (1997). VNS je založený na myšlienke **systematickej zmeny susedstiev** (neighborhoods) — lokálne optimum v jednom susedstve nemusí byť lokálne optimum v inom susedstve.

Princíp

```
   Neighborhood 1          Neighborhood 2          Neighborhood 3
┌─────────────────┐     ┌─────────────────┐     ┌─────────────────┐
│    ●            │     │            ●    │     │    ●            │
│   / \     ●     │     │   ●   / \       │     │   / \   ●       │
│  /   \   /      │ ──→ │    \ /   \      │ ──→ │  /   \ / \      │
│ ●     ●●        │     │     ●     ●     │     │●     ●   ●      │
│ lokálne opt.    │     │                 │     │                 │
└─────────────────┘     └─────────────────┘     └─────────────────┘
  Zaseknutý?              Zmena susedstva!        Nové lokálne opt.
```

Základný VNS (BVNS)

```
1.  Inicializácia: x ← počiatočné riešenie, k_max ← počet susedstiev
2.  Opakuj:
3.    k ← 1
4.    Kým k ≤ k_max:
5.      x' ← Shake(x, k)          // Náhodný bod v k-tom susedstve
6.      x'' ← LocalSearch(x')      // Lokálne prehľadávanie
7.      Ak f(x'') < f(x):          // Ak zlepšenie
8.        x ← x'', k ← 1          // Akceptuj, reštartuj susedstvá
9.      Inak:
10.       k ← k + 1                // Ďalšie susedstvo
11. Kým nie je splnené stop kritérium
```

Prečo je VNS efektívny pre selekciu príznakov?

- **Swap neighborhood** — jemné úpravy (1 príznak)
- **Multi-flip** — väčšie zmeny (viacero príznakov)
- **Add/Remove** — zmena kardiliality výberu
- Kombinácia jemných a hrubých zmien umožňuje **efektívne preskočiť lokálne optimá**

5.3 Simulated Annealing (SA)

Navrhli Kirkpatrick, Gelatt a Vecchi (1983). SA je inšpirovaný procesom **žíhania v metalurgii** — kov sa zohreje na vysokú teplotu a pomaly ochladzuje, čím kryštalická mriežka nájde stav s minimálnou energiou.

Analógia

Metalurgia	Optimalizácia
Stav kryštálu	Riešenie (podmnožina príznakov)
Energia	Fitness funkcia (záporná → minimalizujeme)
Teplota	Parameter T — kontroluje akceptáciu
Chladnutie	Znižovanie T → menej akceptovaných zhoršení

Metalurgia	Optimalizácia
Žíhanie (reheating)	Zvýšenie T pri stagnácii

Boltzmannovo akceptačné kritérium Kľúčová vlastnosť SA — akceptuje aj **horšie riešenia** s pravdepodobnosťou:

$$P(\text{accept}) = \begin{cases} 1 & \text{ak } \Delta E \leq 0 \text{ (zlepšenie)} \\ e^{-\Delta E/T} & \text{ak } \Delta E > 0 \text{ (zhoršenie)} \end{cases}$$

kde $\Delta E = f(x') - f(x)$ a T je aktuálna teplota.

Na začiatku (T vysoké): $P \approx 1$ → akceptuje takmer všetko → **explorácia**
Na konci (T nízke): $P \approx 0$ → akceptuje len zlepšenia → **exploitácia**

Chladiaci rozvrh (Cooling Schedule) Geometrické chladnutie:

$$T_{i+1} = \alpha \cdot T_i, \quad \alpha \in (0, 1)$$

V projekte s predvolenou hodnotou $\alpha = 0.95$: - Po 10 iteráciách: $T = 0.60$ - Po 50 iteráciách: $T = 0.077$ - Po 100 iteráciách: $T = 0.006$

Reheating mechanizmus Ak algoritmus stagnuje (žiadne zlepšenie po `reheat_interval` iteráciách), teplota sa čiastočne zvýši:

$$T_{reheat} = T_{initial} \cdot 0.5$$

a algoritmus sa reštartuje z najlepšieho nájdeného riešenia. Toto zabraňuje predčasnému zaseknutiu.

5.4 Porovnanie VNS a SA

Aspekt	VNS	SA
Filozofia	Mením typ prehľadávania	Mením mieru akceptácie
Explorácia	Cez rôzne susedstvá	Cez teplotu
Deterministickosť	Deterministická akceptácia (len lepšie)	Stochastická akceptácia
Konvergencia	Rýchlejšia (menej iterácií)	Teoretická garancia optimality (ak T→0 dostatočne pomaly)
Parametre	k_max (jednoduché)	T_0, α, T_min, reheat (viac ladenia)

Aspekt	VNS	SA
Vhodnosť	Keď poznáme štruktúru susedstiev	Pre hladké krajiny fitness

5.5 Fitness funkcia v projekte

Oba algoritmy (VNS aj SA) optimalizujú rovnakú fitness funkciu:

$$F(S) = \alpha \cdot \text{Relevancia}(S) + \beta \cdot \text{Synergia}(S) - \gamma \cdot \text{Redundancia}(S)$$

kde:

$$\text{Relevancia}(S) = \frac{1}{|S|} \sum_{f \in S} MI(f, y)$$

$$\text{Synergia}(S) = \frac{1}{\binom{|S|}{2}} \sum_{f_i, f_j \in S} \frac{1}{1 + MI(f_i, f_j)}$$

$$\text{Redundancia}(S) = \frac{1}{\binom{|S|}{2}} \sum_{f_i, f_j \in S} MI(f_i, f_j)$$

Váhy α, β, γ kontrolujú trade-off: - **α = 0.7** (relevancia) — preferuj features s vysokou MI s targetom - **β = 0.2** (synergia) — preferuj features, ktoré sú navzájom rôzne - **γ = 0.3** (redundancia) — penalizuj features, ktoré nesú rovnakú informáciu

---

6. Teória informácie a vzájomná informácia

6.1 Shannonova entropia

Claude Shannon (1948) definoval **entropiu** ako mieru neistoty náhodnej premennej:

$$H(X) = -\sum_{x \in \mathcal{X}} p(x) \log_2 p(x)$$

- $H(X) = 0$ → premenná je deterministická (žiadna neistota)
- $H(X) = \log_2 |\mathcal{X}|$ → maximálna neistota (rovnomerné rozdelenie)

Entropia sa meria v **bitoch** (logaritmus so základom 2) alebo **natoch** (prirodzený logaritmus).

6.2 Podmienená entropia

Entropia X po znalosti Y:

$$H(X|Y) = -\sum_y p(y) \sum_x p(x|y) \log_2 p(x|y)$$

$H(X|Y) \leq H(X)$ — znalosť Y nemôže zvýšiť neistotu o X.

6.3 Vzájomná informácia (Mutual Information)

MI meria **spoločnú informáciu** medzi dvoma premennými:

$$MI(X;Y) = H(X) - H(X|Y) = H(Y) - H(Y|X) = H(X) + H(Y) - H(X,Y)$$

Ekvivalentne cez KL divergenciu:

$$MI(X;Y) = D_{KL}(p(x,y) \| p(x)p(y)) = \sum_{x,y} p(x,y) \log \frac{p(x,y)}{p(x)p(y)}$$

Vlastnosti MI: - $MI(X;Y) \geq 0$ (vždy nezáporná) - $MI(X;Y) = 0 \iff X$ a Y sú nezávislé - $MI(X;X) = H(X)$ (MI so sebou = entropia) - Zachytáva **nelineárne** aj lineárne závislosti - Na rozdiel od korelácie: $r = 0 \not\Rightarrow MI = 0$

Vennov diagram

```
┌──────────────────────────────────────┐
│            H(X, Y)                   │
│   ┌──────────┬──────────┐            │
│   │ H(X|Y)   │  H(Y|X)  │            │
│   │          │          │            │
│   │   ------+------     │            │
│   │  MI(X;Y)            │            │
│   │   ------+------     │            │
│   │          │          │            │
│   └──────────┴──────────┘            │
└──────────────────────────────────────┘
```

6.4 Odhad MI pre spojité premenné — KSG algoritmus

Pre **diskrétne** premenné je MI priamočaro spočítateľná z frekvenčných tabuliek. Pre **spojité** premenné to však vyžaduje odhad distribúcií, čo je problematické:

Metóda	Problém
Histogramový odhad	Citlivý na výber bin size
Kernel Density Estimation	Výpočtovo náročný $O(n^2)$
KSG (Kraskov-Stögbauer-Grassberger)	**Neparametrický, efektívny**

KSG algoritmus (2004) KSG estimátor odhaduje MI z **k-najbližších susedov** v spoločnom priestore (X, Y):

$$\hat{MI}_{KSG}(X;Y) = \psi(k) - \langle \psi(n_x + 1) + \psi(n_y + 1) \rangle + \psi(N)$$

kde: - ψ — digamma funkcia: $\psi(n) = \frac{d}{dn} \ln \Gamma(n)$ - k — počet najbližších susedov (typicky 3-7) - n_x — počet bodov v marginom intervale $|X_j - X_i| \leq \epsilon_i$ pre bod i - n_y — analogicky pre Y - ϵ_i — vzdialenosť k k-tému susedovi v maximovej (Chebyshevovej) norme - $\langle \cdot \rangle$ — priemer cez všetky body

Chebyshevova vzdialenosť KSG používa **maximovú normu** (L∞) namiesto euklidovskej:

$$d_\infty((x_1, y_1), (x_2, y_2)) = \max(|x_1 - x_2|, |y_1 - y_2|)$$

Výhoda: marginálne intervaly sú obdĺžniky, nie kruhy → jednoduchšie počítanie.

6.5 Information Gain

Information Gain (IG) meria, koľko informácie príznak A prinesie o targete T:

$$IG(T, A) = H(T) - H(T|A) = H(T) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} H(S_v)$$

Vyžaduje **diskretizáciu** spojitých príznakov (preto je nutný Binner procesor).

---

7. Testovanie webových aplikácií — Playwright

7.1 Problematika testovania webových aplikácií

Testovanie webových aplikácií je výrazne komplexnejšie než testovanie tradičného softvéru, pretože web aplikácie sú inherentne **asynchrónne, distribuované a nedeterministické**:

Výzva	Popis
Asynchrónnosť	Sieťové požiadavky, WASM inicializácia, animácie
Rôzne prehliadače	Chrome, Firefox, Safari — rôzne enginy
DOM manipulácia	Dynamicky generované elementy, SPA navigácia
State management	Aplikácia má vnútorný stav, ktorý mení UI
Časovanie	Race conditions medzi UI a výpočtami

Pre **WASM aplikácie** pribudá ďalšia vrstva zložitosti: - WASM modul sa načítava asynchrónne - Výpočty v WASM blokujú hlavné vlákno - Dáta prechádzajú cez JS ↔ WASM bridge (serializácia/deserializácia)

7.2 Úrovne testovania

```
┌──────────────────────┐
│    E2E / UI testy    │      ← Playwright (táto aplikácia)
│  (celá aplikácia)    │
├──────────────────────┤
│  Integračné testy    │      ← Kombinácia komponentov
│   (API, pipeline)    │
├──────────────────────┤
│  Jednotkové testy    │      ← Rust #[test], wasm-bindgen-test
│(izolované funkcie)   │
└──────────────────────┘
```

Projekt využíva primárne **E2E testy**, ktoré testujú celú aplikáciu od UI po WASM výpočty.

7.3 Prečo Playwright?

Playwright (Microsoft, 2020) je moderný framework pre end-to-end testovanie webových aplikácií. V porovnaní s alternatívami:

Vlastnosť	Playwright	Selenium	Cypress	Puppeteer
Auto-wait	Ano, automaticky	Nie, manuálne waits	Ciastocne	Nie, manuálne
Multi-browser	Ano (Chromium, Firefox, WebKit)	Ano (všetky)	Nie (len Chromium)	Nie (len Chromium)
Jazyky	Python, JS, Java, C#	Mnohé	Len JS/TS	Len JS
Rýchlosť	Rýchly (CDP + vlastný protocol)	Pomalší (WebDriver)	Rýchly	Rýchly
Paralelizácia	Natívna	Grid	Obmedzená	Manuálna
Network interception	Ano	Nie	Ano	Ano

Vlastnosť	Playwright	Selenium	Cypress	Puppeteer
Izolácia kontextov	Browser contexts	Nové okno	Nie	Incognito
Tracing & debugging	Trace viewer	Screenshot	Dashboard	Nie

Kľúčové výhody Playwright pre túto aplikáciu **1. Auto-waiting mechanizmus**

Playwright automaticky čaká na splnenie podmienok pred interakciou s elementom:

```
# Playwright automaticky čaká, kým element:
# 1. Existuje v DOM
# 2. Je viditeľný
# 3. Je stabilný (nepohybuje sa)
# 4. Je interaktívny (enabled)
page.click("#buildPipelineBtn")  # Žiadne manuálne waits!
```

Toto je kritické pre WASM aplikáciu, kde UI sa aktualizuje asynchrónne po výpočtoch.

2. `wait_for_function` — čakanie na WASM

```
# Čakáme kým WASM modul inicializuje a naplní DOM
page.wait_for_function(
    "document.getElementById('modelsInfo').children.length > 0",
    timeout=30000
)
```

Playwright dokáže čakať na **ľubovoľnú JavaScript podmienku**, čo je ideálne pre WASM aplikácie, kde potrebujeme vedieť, kedy sú výpočty dokončené.

3. Pytest integrácia

Playwright sa natívne integruje s **pytest** cez `pytest-playwright`, čo umožňuje: - Fixture-based setup/teardown - Parametrizované testy - Paralelné spúšťanie - Reporting a filtering

7.4 Implementácia testov v projekte

Testová architektúra

```
tests/
├── conftest.py              # Fixtures + server management
│   ├── server()             # Auto-start/stop HTTP servera
│   ├── app_page()           # Navigácia + WASM init
│   ├── loaded_page()        # Kompletný setup (pipeline + dáta)
│   └── SAMPLE_CSV           # Testovací dataset
├── test_01_init.py          # WASM inicializácia
├── test_02_data_loading.py  # Načítanie dát
├── test_03_pipeline.py      # Build pipeline
├── test_04_target_analysis.py # Analýza targetu
```

```
├── test_05_selectors.py     # Selektory príznakov
├── test_06_heatmap.py       # Vizualizácia heatmapy
└── test_07_editor.py        # Editor dát
```

Fixture hierarchy (pyramída závislostí)

```
server                    ← session-scoped, 1× za celý test run
  └── app_page            ← per-test, navigácia + WASM init
        └── loaded_page   ← per-test, pipeline + data + target
```

server fixture automaticky: 1. Skontroluje, či server už beží na porte 3333 2. Ak nie, spustí `serve.sh` ako subprocess 3. Čaká (max 15s) na dostupnosť portu 4. Po testoch ukončí server proces (SIGTERM na process group)

`loaded_page` fixture simuluje kompletný user flow: 1. Otvorí stránku a čaká na WASM 2. Vyberie model (logreg) a buildne pipeline 3. Vloží testovací CSV dataset 4. Vyberie target stĺpec (approved) 5. Potvrdí target a čaká na spracovanie

Vzory testovania (Test patterns) Pattern 1: Smoke test — existencia elementov

```python
def test_models_displayed(app_page):
    """Available models are populated after WASM init."""
    models_div = page.locator("#modelsInfo")
    count = models_div.locator(".option-item, span, div").count()
    assert count >= 2, "Expected at least 2 models"
```

Pattern 2: User flow — kompletný scenár

```python
def test_build_pipeline_knn(app_page):
    """Build KNN pipeline and check K parameter UI."""
    page.select_option("#modelSelect", "knn")
    page.wait_for_selector("#knnKGroup", state="visible")
    page.fill("#knnK", "7")
    page.click("#buildPipelineBtn")
    page.wait_for_selector("#pipelineStatus", timeout=10000)
    status = page.inner_text("#pipelineStatus")
    assert "error" not in status.lower()
```

Pattern 3: Async WASM výpočet — čakanie na výsledok

```python
def test_compare_selectors(loaded_page):
    """Compare feature selectors produces results."""
    page.click("#compareSelectorsBtn")
    page.wait_for_selector(".selector-result", timeout=60000)
    results = page.locator(".selector-result").count()
    assert results >= 1
```

Pattern 4: Modal interakcia — open/close/verify

```python
def test_heatmap_escape_closes(loaded_page):
    """Pressing Escape closes the heatmap modal."""
```

```
    page.click("#showMatrixBtn")
    page.wait_for_selector("#heatmapModal.show")
    page.keyboard.press("Escape")
    page.wait_for_selector("#heatmapModal.show", state="hidden")
```

Pattern 5: Console error monitoring

```
def test_wasm_no_errors(page, server):
    """WASM loads without JS console errors."""
    errors = []
    page.on("console", lambda msg: errors.append(msg.text)
            if msg.type == "error" else None)
    page.goto(server, wait_until="networkidle")
    # Filter non-critical errors
    critical = [e for e in errors if "favicon" not in e.lower()]
    assert len(critical) == 0
```

7.5 Pokrytie testov

Oblasť	Testy	Čo sa overuje
Inicializácia	7	WASM modul sa načíta, UI sa naplní, žiadne JS chyby
Dáta	9	CSV parsovanie, target výber, inspect, split slider
Pipeline	7	Každý model sa dá buildnúť, parametre, eval mode
Target analýza	5	Analyzer karty, porovnanie, detail
Selektory	5	Selector karty, select all, porovnanie
Heatmapa	12	Modal, matice, tooltip, legenda, sort, escape
Editor	4	Otvorenie, stĺpce, procesory
Celkom	**49**	

---

8. Architektúra systému

Štruktúra projektu

```
src/
├── lib.rs                    # Vstupný bod WASM modulu
├── mi_estimator.rs           # Zdieľaný KSG MI estimátor
├── data_loading/             # Načítavanie dát (CSV, JSON)
│   ├── csv_data_loader.rs
│   ├── json_data_loader.rs
│   ├── factory.rs
│   └── mod.rs
```

```
├── models/                   # ML modely
│   ├── IModel.rs             # Trait definícia
│   ├── linreg.rs             # Lineárna regresia
│   ├── logreg.rs             # Logistická regresia
│   ├── knn.rs                # K-Nearest Neighbors
│   ├── tree.rs               # Decision Tree
│   └── factory.rs
├── processing/               # Preprocessing procesorov
│   ├── scaler.rs             # Standard Scaler
│   ├── minmax_scaler.rs      # MinMax Scaler
│   ├── robust_scaler.rs      # Robust Scaler
│   ├── binner.rs             # Diskretizácia
│   ├── ohencoder.rs          # One-Hot Encoding
│   ├── label_encoder.rs      # Label Encoding
│   ├── null_handler.rs       # Ošetrenie chýbajúcich hodnôt
│   ├── outlier_clipper.rs    # Orezávanie odľahlých hodnôt
│   ├── log_transformer.rs    # Log transformácia
│   ├── power_transformer.rs  # Box-Cox / Yeo-Johnson
│   ├── time_converter.rs     # Konverzia času
│   ├── selective_processor.rs # Automatická detekcia typov stĺpcov
│   ├── processor_decorator.rs # Reťazenie procesorov
│   └── factory.rs
├── feature_selection_strategies/  # Selektory príznakov
│   ├── variance_selector.rs
│   ├── correlation_selector.rs
│   ├── chi_square_selector.rs
│   ├── information_gain_selector.rs
│   ├── mutual_information_selector.rs
│   ├── smc_selector.rs
│   ├── synergy_vns_selector.rs    # VNS metaheuristika
│   ├── synergy_sa_selector.rs     # SA metaheuristika
│   └── factory.rs
├── target_analysis/          # Analýza cieľovej premennej
│   ├── correlation_analyzer.rs
│   ├── entropy_analyzer.rs
│   ├── mutual_information_analyzer.rs
│   ├── smc_analyzer.rs
│   └── factory.rs
├── embedded/                 # Embedded selektory
│   ├── random_forest_selector.rs
│   ├── ridge_selector.rs
│   └── factory.rs
├── evaluation/               # Evaluácia modelov
│   ├── evaluator.rs
│   └── metrics.rs
├── pipeline/                 # Jadro pipeline
│   ├── pipeline.rs           # Fasáda (Facade)
│   ├── builder.rs            # Builder pattern
```

```
│    ├── director.rs           # Presety
│    └── compatibility.rs      # Kompatibilita komponentov
└── wasm_api/                  # WASM API vrstva
     ├── ml_pipeline_api.rs    # Hlavný API endpoint (~3000 riadkov)
     ├── factory_api.rs        # Factory API
     └── data_loader_api.rs    # Data Loader API
```

Vrstvy systému

```
┌──────────────────────────────────────────────────────────┐
│                    WASM API vrstva                       │
│  WasmMLPipeline · WasmFactory · WasmDataLoader           │
│  (serde-wasm-bindgen, JsValue konverzie)                 │
├──────────────────────────────────────────────────────────┤
│                   Pipeline vrstva                        │
│  MLPipeline (Facade) · MLPipelineBuilder · Director      │
│  CompatibilityRegistry (Singleton)                       │
├──────────────────────────────────────────────────────────┤
│              Doménová logika                             │
│  Modely · Procesory · Selektory · Analyzéry              │
│  (Strategy pattern — trait implementácie)                │
├──────────────────────────────────────────────────────────┤
│             Spoločné utility                             │
│  MI Estimátor (KSG + KD-tree) · Evaluátor                │
│  DataLoader Factory · ProcessorChain                     │
└──────────────────────────────────────────────────────────┘
```

---

9. Návrhové vzory

Projekt využíva **6 klasických návrhových vzorov** (Design Patterns):

9.1 Builder Pattern — `MLPipelineBuilder`

Umožňuje postupné skladanie pipeline s fluent API:

```rust
let pipeline = MLPipelineBuilder::new()
    .model("knn")                        // Výber modelu
    .model_param("k", "5")               // Parameter modelu
    .processor("scaler")                 // Preprocessing
    .feature_selector("variance")        // Selekcia príznakov
    .selector_param("threshold", "0.01") // Parameter selektora
    .evaluation_mode("classification")   // Mód evaluácie
    .build()?;                           // Zostavenie + validácia
```

Validácia pri `build()`: 1. Overí, že model je nastavený 2. Skontroluje kompatibilitu cez `CompatibilityRegistry` 3. Vytvorí komponenty cez príslušné Factory triedy 4. Automaticky detekuje evaluation mode, ak nie je zadaný

9.2 Factory Pattern

Každá skupina komponentov má vlastnú Factory:

Factory	Metóda	Vstup → Výstup
`ModelFactory`	`create(name)`	`"knn"` → `Box<dyn IModel>`
`ProcessorFactory`	`create(name)`	`"scaler"` → `Box<dyn DataProcessor>`
`FeatureSelectorFactory`	`create(name)`	`"variance"` → `Box<dyn FeatureSelector>`
`TargetAnalyzerFactory`	`create(name)`	`"correlation"` → `Box<dyn TargetAnalyzer>`
`DataLoaderFactory`	`create(format)`	`"csv"` → `Box<dyn DataLoader>`
`EmbeddedSelectorFactory`	`create(name)`	`"ridge"` → `Box<dyn EmbeddedFeatureSelector>`

Všetky Factory triedy poskytujú aj metódy `available()`, `get_description()` a `get_supported_params()`.

9.3 Strategy Pattern

Správanie je zapuzdrené za spoločným trait rozhraním. Klient pracuje s `Box<dyn Trait>` — konkrétna implementácia je zameniteľná:

```rust
pub trait IModel {
    fn get_name(&self) -> &str;
    fn train(&mut self, x: &DenseMatrix<f64>, y: &[f64]) -> Result<(), String>;
    fn predict(&self, x: &DenseMatrix<f64>) -> Result<Vec<f64>, String>;
    fn get_supported_params(&self) -> Vec<&str>;
    fn set_param(&mut self, key: &str, value: &str) -> Result<(), String>;
}
```

Rovnako pre `DataProcessor`, `FeatureSelector`, `TargetAnalyzer`, `DataLoader`, `EmbeddedFeatureSelector`.

9.4 Facade Pattern — `MLPipeline`

`MLPipeline` je hlavný vstupný bod, ktorý orchestruje celý ML workflow:

```
Vstupná matica X, vektor y
        │
        ▼
┌─── Feature Selection ──┐
│   Výber relevantných    │
│   príznakov             │
└─────────────────────────┘
            │
            ▼
┌─── Preprocessing ──────┐
│   Fit + Transform na    │
```

```
│   trénovacej sade          │
└──────────────────────────┘
            │
            ▼
┌─── Model Training ────────┐
│   Trénovanie modelu        │
│   na predspracovaných dátach│
└──────────────────────────┘
            │
            ▼
┌─── Evaluation ────────────┐
│   Metriky na testovacej    │
│   sade (ak je split)       │
└──────────────────────────┘
```

9.5 Decorator Pattern — `ProcessorChain`

Umožňuje reťazenie viacerých procesorov:

```
// Vytvorí reťazec: Outlier Clipper → Standard Scaler
let chain = ProcessorFactory::create_chain(vec!["outlier_clipper", "scaler"]);
```

`SelectiveProcessor` je ďalší dekorátor, ktorý automaticky detekuje typy stĺpcov (Numeric, Categorical, Discrete) a aplikuje procesor len na vhodné stĺpce.

9.6 Singleton Pattern — `CompatibilityRegistry`

Jediná inštancia registra kompatibility, chránená `Mutex`-om:

```
static REGISTRY: Lazy<Mutex<CompatibilityRegistry>> = Lazy::new(|| {
    Mutex::new(CompatibilityRegistry::new())
});
```

Definuje, ktoré modely sú kompatibilné s ktorými procesormi a selektormi.

---

10. Modely strojového učenia

Všetky modely implementujú trait `IModel` a používajú knižnicu **SmartCore**.

10.1 Lineárna Regresia (`linreg`)

Vlastnosť	Hodnota
Typ	Regresia
Implementácia	`smartcore::linear::linear_regression::Linea`
Parametre	`solver`: voľba riešiča — `"qr"` (QR dekompozícia) alebo `"svd"` (SVD dekompozícia)
Predvolená hodnota	`solver = "qr"`

Vlastnosť	Hodnota
Popis	Predikuje spojité hodnoty minimalizáciou súčtu štvorcov reziduálov

Matematický model:

$$\hat{y} = X \cdot \beta + \epsilon$$

kde $\beta = (X^T X)^{-1} X^T y$ (metóda najmenších štvorcov).

10.2 Logistická Regresia (`logreg`)

Vlastnosť	Hodnota
Typ	Klasifikácia
Implementácia	`smartcore::linear::logistic_regression::Lo`
Parametre	`alpha`: regularizačný parameter (L2 penalizácia, f64)
Predvolená hodnota	`alpha = 1.0`
Popis	Binárna klasifikácia modelujúca pravdepodobnosť triedy

Sigmoid funkcia:

$$P(y = 1|x) = \frac{1}{1 + e^{-(\beta_0 + \beta^T x)}}$$

Cieľová premenná y sa automaticky konvertuje na u32 triedy.

10.3 K-Nearest Neighbors (`knn`)

Vlastnosť	Hodnota
Typ	Regresia aj klasifikácia
Implementácia	`smartcore::neighbors::knn_regressor::KNNRe`
Parametre	k: počet susedov (usize)
Predvolená hodnota	`k = 5`
Popis	Predikuje priemer k najbližších susedov (euklidovská vzdialenosť)

Vzdialenostná metrika:

$$d(x, x') = \sqrt{\sum_{i=1}^{n} (x_i - x'_i)^2}$$

10.4 Decision Tree (`tree`)

Vlastnosť	Hodnota
Typ	Regresia aj klasifikácia
Implementácia	`smartcore::tree::decision_tree_regressor::`
Parametre	`max_depth` (u16), `min_samples_split` (u16)
Predvolené hodnoty	`max_depth = 10`, `min_samples_split = 2`
Popis	Rekurzívne delí priestor príznakov podľa najlepšieho rozdeľovacieho kritéria

---

11. Procesory dát (preprocessing)

Procesory implementujú trait `DataProcessor` a sú automaticky obalené v `SelectiveProcessor`, ktorý detekuje typ stĺpca:

- **Numeric** — stĺpce s reálnymi číslami (≤ 20 unikátnych hodnôt = Discrete)
- **Categorical** — stĺpce s textovými hodnotami
- **Discrete** — stĺpce s malým počtom unikátnych číselných hodnôt

11.1 Numerické procesory

Standard Scaler (`scaler`) Normalizácia na priemer 0 a štandardnú odchýlku 1:

$$x_{scaled} = \frac{x - \mu}{\sigma}$$

Parameter	Typ	Popis
—	—	Žiadne konfigurovateľné parametre

MinMax Scaler (`minmax_scaler`) Normalizácia do intervalu [min, max]:

$$x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}} \cdot (max - min) + min$$

Parameter	Typ	Predvolená hodnota	Popis
`min`	f64	0.0	Dolná hranica normalizácie
`max`	f64	1.0	Horná hranica normalizácie

Robust Scaler (`robust_scaler`) Škálovanie pomocou mediánu a IQR, odolné voči odľahlým hodnotám:

$$x_{scaled} = \frac{x - \text{medián}}{IQR}$$

kde $IQR = Q_3 - Q_1$ (interkvartilové rozpätie).

Outlier Clipper (`outlier_clipper`) Orezáva odľahlé hodnoty. Podporuje tri metódy: - **IQR metóda** — orezáva mimo $[Q_1 - 1.5 \cdot IQR,\ Q_3 + 1.5 \cdot IQR]$ - **Percentilová metóda** — orezáva mimo zadaného percentilu - **Z-score metóda** — orezáva hodnoty s |z| > prahová hodnota

Log Transformer (`log_transformer`) Logaritmická transformácia pre normalizáciu rozdelenia:

$$x_{transformed} = \ln(x + \text{offset})$$

Parameter	Typ	Predvolená hodnota	Popis
`offset`	f64	1.0	Konštanta pridaná pred logaritmom (pre nulové hodnoty)

Power Transformer (`power_transformer`) Box-Cox alebo Yeo-Johnson transformácia na normalizáciu rozdelenia.

11.2 Diskretizačné procesory

Binner (`binner`) Diskretizuje spojité hodnoty do binov (intervalov):

Parameter	Typ	Predvolená hodnota	Popis
`bins`	usize	10	Počet diskretizačných intervalov

Dôležité: Vyžadovaný pre `Information Gain` selektor.

11.3 Kódovacie procesory (pre kategorické dáta)

One-Hot Encoder (`onehot`) Vytvára binárne stĺpce pre každú kategóriu. Napr. farba ∈ {červená, modrá, zelená} → 3 binárne stĺpce.

Label Encoder (`label_encoder`) Kóduje kategórie na celé čísla: A → 0, B → 1, C → 2.

Ordinal Encoder (`ordinal_encoder`) Ako Label Encoder, ale zachováva poradie hodnôt.

Frequency Encoder (`frequency_encoder`) Kóduje kategórie podľa frekvencie výskytu v dátach.

Target Encoder (`target_encoder`) Kóduje kategórie priemerom cieľovej premennej pre danú kategóriu.

11.4 Špeciálne procesory

Null Handler (`null_handler`) Ošetruje chýbajúce hodnoty (NaN, null, prázdne reťazce):

Parameter	Typ	Popis
`null_repr`	String	Reprezentácia null hodnôt (predvolená: `""`)
`strategy`	String	Stratégia nahradenia: `"mean"`, `"median"`, `"mode"`

Time Converter (`time_converter`) Konvertuje časové hodnoty medzi formátmi (sekundy, minúty, hodiny).

Comma to Dot (`comma_to_dot`) Nahrádza desatinné čiarky bodkami (`"3,14"` → `"3.14"`).

Thousands Separator Remover (`thousands_separator_remover`) Odstraňuje oddeľovače tisícov (`"1,000,000"` → `"1000000"`).

11.5 Reťazenie procesorov

Procesory je možné reťaziť pomocou `ProcessorChain` (Decorator pattern):

```
let chain = ProcessorFactory::create_chain(vec![
    "null_handler",
    "outlier_clipper",
```

```
    "scaler"
]);
// Výsledok: NullHandler → OutlierClipper → StandardScaler
```

---

12. Selektory príznakov (feature selection)

Feature selection je **kľúčový krok** v ML pipeline, ktorý redukuje počet príznakov na najrelevantnejšie.

Prehľad selektorov

Selektor	Typ	Popis	Metrika
Variance Threshold	Filter	Odstráni konštantné features	Variancia
Correlation	Filter	Výber podľa korelácie s targetom	Pearsonova korelácia
Chi-Square Test	Filter	Test nezávislosti (len klasifikácia)	χ^2 štatistika
Information Gain	Filter	Redukcia entropie (vyžaduje Binner!)	IG/Entropia
Mutual Information	Filter	KSG estimátor (kontinuálne dáta)	MI score
SMC	Filter	Príspevok k R^2 predikcii	R^2 drop
Synergy VNS	**Wrapper (metaheuristika)**	**Variable Neighborhood Search**	**Fitness**
Synergy SA	**Wrapper (metaheuristika)**	**Simulated Annealing**	**Fitness**

12.1 Variance Threshold

Odstráni príznaky s varianciou pod prahovou hodnotou — typicky konštantné alebo takmer konštantné stĺpce.

Parameter	Predvolená	Popis
`threshold`	0.01	Minimálna variancia

12.2 Correlation Selector

Vyberie príznaky s najvyššou absolútnou Pearsonovou koreláciou k cieľovej premennej.

Parameter	Predvolená	Popis
`threshold`	0.95	Prahová hodnota korelácie

12.3 Chi-Square Test

Test nezávislosti medzi príznakmi a targetom. **Len pre klasifikáciu.**

$$\chi^2 = \sum \frac{(O_i - E_i)^2}{E_i}$$

Parameter	Predvolená	Popis
`num_features`	5	Počet vybraných príznakov

12.4 Information Gain

Meria redukciu entropie — koľko informácie príznak prinesie o targete.

$$IG(T, A) = H(T) - H(T|A)$$

Upozornenie: Vyžaduje predchádzajúce spracovanie procesorom `Binner`!

Parameter	Predvolená	Popis
`num_features`	5	Počet vybraných príznakov

12.5 Mutual Information (KSG)

Odhaduje vzájomnú informáciu medzi príznakom a targetom pomocou KSG estimátora. Funguje na **spojitých dátach** bez nutnosti diskretizácie.

Parameter	Predvolená	Popis
`num_features`	5	Počet vybraných príznakov

12.6 SMC (Squared Multiple Correlation)

Meria príspevok každého príznaku k multi-lineárnej predikcii targetu cez pokles v R^2:

$$SMC_i = R^2_{all} - R^2_{all \setminus i}$$

Parameter	Predvolená	Popis
`num_features`	5	Počet vybraných príznakov

12.7 Synergy VNS (Variable Neighborhood Search)

Metaheuristický selektor, ktorý optimalizuje **kombinovanú fitness funkciu** prehľadávaním rôznych typov susedstiev.

Fitness funkcia

$$F(S) = \alpha \cdot \overline{MI}(S) + \beta \cdot \text{Diverzita}(S) - \gamma \cdot \text{Redundancia}(S)$$

kde: - $\overline{MI}(S)$ — priemerná MI vybraných features s targetom (relevancia) - Diverzita(S) — priemerná inverzná MI medzi features: $\frac{1}{1+MI(f_i,f_j)}$ (synergia) - Redundancia(S) — priemerná MI medzi features (penalizácia)

Typy susedstiev (neighborhoods)

k	Typ	Popis
1	Swap	Vymeň 1 príznak za iný
2	Flip-k	Vymeň k príznakov naraz (k=2)
3	Add	Pridaj 1 príznak
4	Remove	Odober 1 príznak

Parametre

Parameter	Predvolená	Popis
`num_features`	5	Cieľový počet príznakov
`max_iterations`	100	Počet iterácií algoritmu
`k_max`	4	Počet typov neighborhoods (1-4)
`alpha`	0.7	Váha relevancie (MI s targetom)
`beta`	0.2	Váha synergie (diverzita features)
`gamma`	0.3	Penalizácia redundancie
`initial_solution`	`"greedy"`	Počiatočné riešenie (`"greedy"` / `"random"`)

Algoritmus VNS

```
1. Inicializácia: S₀ = greedy/random riešenie
2. Pre každú iteráciu:
   a. k = 1
   b. Kým k ≤ k_max:
      i.   Generuj suseda S' v k-tom neighborhood
      ii.  Lokálne prehľadávanie: S'' = local_search(S')
      iii. Ak F(S'') > F(S*): aktualizuj najlepšie, k = 1
      iv.  Inak: k++
3. Diversifikácia: každých 30 iterácií reštart
```

12.8 Synergy SA (Simulated Annealing)

Metaheuristický selektor využívajúci **simulované žíhanie** s **Boltzmannovým akceptačným kritériom**. Na rozdiel od VNS akceptuje aj horšie riešenia s pravdepodobnosťou závislou od teploty, čo mu umožňuje efektívnejšie unikať z lokálnych optím.

Boltzmannovo akceptačné kritérium

$$P(\text{accept}) = \begin{cases} 1 & \text{ak } \Delta F \geq 0 \\ e^{\Delta F / T} & \text{ak } \Delta F < 0 \end{cases}$$

kde $\Delta F = F(S') - F(S)$ je zmena fitness a T je aktuálna teplota.

Chladiaci rozvrh

$$T_{i+1} = T_i \cdot \text{cooling_rate}$$

S **reheating** mechanizmom: po `reheat_interval` iteráciách bez zlepšenia sa teplota čiastočne zvýši a algoritmus sa reštartuje z najlepšieho riešenia.

Parametre

Parameter	Predvolená	Popis
`num_features`	5	Cieľový počet príznakov
`max_iterations`	200	Maximálny počet iterácií
`initial_temp`	1.0	Počiatočná teplota (vyššia = viac explorácie)
`cooling_rate`	0.95	Rýchlosť chladnutia (0.5 – 0.9999)
`min_temp`	0.001	Zastavenie pod touto teplotou

Parameter	Predvolená	Popis
`reheat_interval`	50	Po koľkých iteráciách reheating
`alpha`	0.7	Váha relevancie (MI s targetom)
`beta`	0.2	Váha synergie
`gamma`	0.3	Penalizácia redundancie
`initial_solution`	`"greedy"`	Počiatočné riešenie (`"greedy"` / `"random"`)

Porovnanie VNS vs SA

Aspekt	VNS	SA
Prehľadávanie	Deterministické zmeny neighborhoods	Stochastické, pravdepodobnostné
Akceptačné kritérium	Len lepšie riešenia	Aj horšie (Boltzmann)
Únik z lokálnych optím	Cez zmenu susedstva	Cez teplotu → akceptovanie horších
Parametrizácia	k_max, neighborhoods	Teplota, cooling_rate, reheating
Konvergencia	Typicky rýchlejšia	Pomalšia ale dôkladnejšia

---

13. Analýza cieľovej premennej (target analysis)

Analýza pomáha používateľovi identifikovať **najvhodnejší stĺpec ako cieľovú premennú**.

13.1 Correlation Analyzer

Vypočíta sumu štvorcov Pearsonových korelácií medzi kandidátom a ostatnými stĺpcami:

$$\text{Score} = \sum_{i=1}^{n} r_i^2$$

Stĺpec s najvyšším Σr^2 je najlepší kandidát — „vysvetľuje” najviac z ostatných premenných.

13.2 Mutual Information Analyzer

Používa KSG estimátor na výpočet sumy MI medzi kandidátom a všetkými ostatnými stĺpcami:

$$\text{Score} = \sum_{i=1}^{n} MI(Y, X_i)$$

Zachytáva aj **nelineárne závislosti**, ktoré korelácia nezachytí.

13.3 SMC Analyzer (Squared Multiple Correlation)

Meria, koľko variability kandidáta je vysveteľných ostatnými premennými pomocou multi-lineárneho modelu.

13.4 Entropy Analyzer

Kombinuje entropiu a Information Gain: - **Entropia** — miera neistoty: $H(Y) = -\sum p_i \log_2 p_i$ - **Information Gain** — redukcia entropie pri znalosti iného príznaku

Výstup analyzérov

Každý analyzér vracia štruktúru `TargetCandidate`:

```rust
pub struct TargetCandidate {
    pub column_index: usize,
    pub column_name: String,
    pub score: f64,
    pub unique_values: usize,
    pub variance: f64,
    pub suggested_type: String,    // "classification" / "regression"
    pub extra_metrics: HashMap<String, f64>,
}
```

Typ (klasifikácia vs regresia) sa automaticky heuristicky odhadne: < 20 unikátnych hodnôt → klasifikácia.

---

14. Embedded selektory

Na rozdiel od filter/wrapper metód, embedded selektory extrahujú dôležitosť príznakov **priamo z trénovaného modelu**.

14.1 Random Forest Selector

Vlastnosť	Hodnota
Podpora	Klasifikácia aj regresia
Metóda	Dôležitosť príznakov zo stromového modelu
Kľúče	`tree`, `random_forest`, `tree_importance`

14.2 Ridge Selector

Vlastnosť	Hodnota
Podpora	Len regresia
Metóda	L2 regularizácia — veľkosť koeficientov indikuje dôležitosť
Kľúče	`ridge`, `l2`
Parameter	`alpha` (regularizačná sila, predvolená: 1.0)

Automatický výber: pre klasifikáciu → `RandomForestSelector`, pre regresiu → `RidgeSelector`.

---

15. Mutual Information — KSG estimátor

Algoritmus KSG (Kraskov-Stögbauer-Grassberger)

Zdieľaný modul `mi_estimator.rs` implementuje **KSG algoritmus** pre odhad vzájomnej informácie medzi dvoma premennými bez nutnosti diskretizácie.

Vzorec

$$MI(X,Y) = \psi(k) - \langle \psi(n_x + 1) + \psi(n_y + 1) \rangle + \psi(N)$$

kde: - ψ — digamma funkcia - k — počet najbližších susedov - n_x, n_y — počet bodov v marginom intervale - N — celkový počet vzoriek

Optimalizácie Modul využíva dve kľúčové optimalizácie pre efektívny výpočet:

1. KD-tree s Chebyshevovou vzdialenosťou

$$d_\infty(a,b) = \max_i |a_i - b_i|$$

KD-tree (`kdtree` crate) umožňuje nájsť k najbližších susedov v $O(k \log n)$ namiesto $O(n)$ pri naivnom prehľadávaní.

2. Zoradené polia + binárne vyhľadávanie

Pre počítanie marginálnych susedov v 1D priestore sa používa zoradený index s binárnym hľadaním. `SortedIndex` štruktúra umožňuje count_within operáciu v $O(\log n)$ namiesto $O(n)$.

Celková zložitosť:

$$O(n \cdot k \cdot \log n) \quad \text{namiesto} \quad O(n^2)$$

Adaptívne k Pre veľké datasety sa automaticky redukuje k: - > 1000 vzoriek: k ≤ 2 - > 500 vzoriek: k ≤ 3 - Inak: k = 3 (predvolené)

API

```
// Jedna dvojica premenných
pub fn estimate_mi_ksg(x: &[f64], y: &[f64], k: usize) -> f64;

// Celá matica (všetky páry stĺpcov)
pub fn compute_mi_matrix(columns: &[Vec<f64>], k: usize) -> Vec<Vec<f64>>;
```

---

16. Evaluácia modelov

`ModelEvaluator` automaticky rozpozná typ úlohy (klasifikácia / regresia) a nasadí príslušné metriky.

16.1 Klasifikačné metriky

Metrika	Vzorec	Popis
Accuracy	$\frac{TP+TN}{TP+TN+FP+FN}$	Podiel správnych predikcií
Precision	$\frac{TP}{TP+FP}$	Presnosť pozitívnych predikcií
Recall	$\frac{TP}{TP+FN}$	Úplnosť zachytenia pozitívnych
F1 Score	$\frac{2 \cdot P \cdot R}{P+R}$	Harmonický priemer precision a recall
Specificity	$\frac{TN}{TN+FP}$	Schopnosť identifikovať negatívne
FPR	$\frac{FP}{FP+TN}$	Miera falošných poplachov
FNR	$\frac{FN}{FN+TP}$	Miera chýbajúcich detekcií

Metrika	Vzorec	Popis
MCC	$\frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$	Matthews Correlation — vhodný pre nevyvážené dáta

16.2 Regresné metriky

Metrika	Vzorec	Popis
MSE	$\frac{1}{n}\sum(y_i - \hat{y}_i)^2$	Stredná kvadratická chyba
RMSE	$\sqrt{MSE}$	Odmocnina MSE
MAE	$\frac{1}{n}\sum\lvert y_i - \hat{y}_i\rvert$	Stredná absolútna chyba
R²	$1 - \frac{SS_{res}}{SS_{tot}}$	Koeficient determinácie
MAPE	$\frac{100}{n}\sum\left\lvert\frac{y_i - \hat{y}_i}{y_i}\right\rvert$	Stredná absolútna percentuálna chyba
MASE	$\frac{MAE}{MAE_{naive}}$	Škálovaná chyba vs naivná predikcia
MdAE	medián$(\lvert y_i - \hat{y}_i\rvert)$	Medián absolútnej chyby
Explained Var	$1 - \frac{Var(y-\hat{y})}{Var(y)}$	Vysvetlená variancia
Pearson r	$\frac{Cov(y,\hat{y})}{\sigma_y \sigma_{\hat{y}}}$	Pearsonova korelácia predikcií

---

17. Pipeline — životný cyklus

17.1 Vytvorenie pipeline

```
Používateľ → buildFromConfig(config) → MLPipelineBuilder
                                        │
                                        ▼
                                  ┌──────────────┐
                                  │ Validácia    │
                                  │ kompatibility│
                                  └──────────────┘
                                        │
                                        │
                                        │
```

```
                    Box<IModel>  Box<Processor>  Box<Selector>
                         │            │             │
                         └────────────┼─────────────┘
                                      ▼
                                  MLPipeline
```

17.2 Trénovanie

```
loadData(csv, target)
        │
        ▼
  ┌─────────────┐      ┌──────────────┐
  │ Parse CSV   │──────→│ X: DenseMatrix│
  │ Extract y   │      │ y: Vec<f64>   │
  └─────────────┘      └──────────────┘
                              │
                              │
                    trainWithSplit(ratio)
                              │
                  ┌───────────┼──────────┐
                  ▼                      ▼
         Train Split (80%)              Test Split (20%)
                  │                      │
     ┌────────────┼──────────┐           │
     ▼                       ▼           │
Feature Selection    (vybrané indexy)     │
     │                       │           │
     ▼                       │           │
Processor.fit()              │           │
     │                       │           │
     ▼                       │           │
Processor.transform()        │           │
     │                       │           │
     ▼                       │           │
Model.train()                │           │
     │                       │           │
     └───────────────────────┼           │
                             ▼           ▼
                    Model.predict(test)  ←── Apply same
                             │               feature indices
                             │               + processor
                             ▼
                     Evaluate(y_true, y_pred)
                             │
                             ▼
                    EvaluationReport
```

17.3 Deterministický split

Pipeline používa **deterministický split** namiesto náhodného. To zabezpečuje: - Rovnaký train/test split pre všetky selektory pri porovnávaní - Reprodukovateľné výsledky - Férové porovnanie metód selekcie

Split indexy sa cachujú v `split_cache` a opätovne používajú.

11.4 Cachovací systém

Cache	Obsah	Invalidácia
`data_cache`	X matica + y vektor	Pri `loadData()`
`analysis_cache`	Parsované CSV stĺpce + hlavičky	Pri `loadData()`
`matrices_cache`	Korelačná + MI + SMC matica	Pri `loadData()`
`split_cache`	Train/test indexy + data splity	Pri `loadData()`

---

18. WASM API rozhranie

Aplikácia exportuje **3 WASM triedy** do JavaScriptu:

18.1 `WasmMLPipeline` — hlavný pipeline

Najväčšia trieda (~3000 riadkov), poskytuje kompletný ML workflow:

Metóda	Popis
`new()`	Vytvorí novú inštanciu pipeline
`buildFromConfig(config)`	Zostaví pipeline z JSON konfigurácie
`buildFromPreset(name, model, ...)`	Zostaví z predpripraveného presetu
`loadData(data, target, format)`	Načíta dáta a pripraví X, y
`train()`	Trénuje model na celom datasete
`trainWithSplit(ratio)`	Trénuje s train/test splitom
`trainWithFeatureIndices(ratio, indices)`	Trénuje s explicitnými feature indexami
`predict(input)`	Predikcia na jednom vstupe
`getInfo()`	Info o aktuálnom pipeline
`inspectData(maxRows)`	Náhľad na surové dáta
`inspectProcessedData(maxRows)`	Náhľad na predspracované dáta
`getFeatureSelectionInfo()`	Detaily o selekcii features
`compareSelectors(data, target, format, selectors)`	Porovnanie selektorov
`analyzeTargetCandidates(data, format)`	Analýza kandidátov na target

Metóda	Popis
`analyzeTargetWith(data, format, method)`	Analýza špecifickou metódou
`compareTargetAnalyzers(data, format, methods)`	Porovnanie analyzérov
`getFeatureMatrices(data, format)`	Korelačná + MI matica pre heatmapu
`checkFeatureRedundancy(...)`	Kontrola redundancie features
`getEditableData(data, format)`	Dáta pre editor
`applyProcessorToColumn(...)`	Aplikácia procesora na stĺpec
`deleteColumn(data, column)`	Zmazanie stĺpca
`setCellValue(...)`	Úprava bunky
`replaceAllInColumn(...)`	Nahradenie hodnôt v stĺpci
`getEmbeddedFeatureRanking(...)`	Embedded feature importance

18.2 `WasmFactory` — factory pre frontend

Metóda	Popis
`new()`	Vytvorí factory
`getAvailableOptions()`	Všetky modely, procesory, selektory, formáty, presety
`getCompatibleProcessors(model)`	Kompatibilné procesory pre model
`getCompatibleSelectors(model)`	Kompatibilné selektory pre model
`getModelParams(model)`	Parametre modelu
`getSelectorParams(selector)`	Parametre selektora
`getProcessorParamDefinitions(proc)`	Detailné definície parametrov (typ, min, max, popis)
`getPresetDetails(preset)`	Detail presetu

18.3 `WasmDataLoader` — načítavač dát

Metóda	Popis
`new(format)`	Loader pre špecifický formát
`createAuto(data)`	Auto-detekcia formátu
`getAvailableColumns(data)`	Zoznam stĺpcov
`validateFormat(data)`	Validácia formátu
`loadData(data, target)`	Načítanie dát s targetom

---

19. Načítavanie dát

Podporované formáty

Formát	Popis	Auto-detekcia
CSV	Comma-Separated Values, štandardný tabuľkový formát	Obsahuje `,` a `\n`
JSON	Array of Objects `[{"col1": val1, ...}, ...]`	Začína `[{`

Proces načítania

```
Vstupný reťazec (CSV/JSON)
        │
        ▼
  DataLoaderFactory::create(format)
        │
        ▼
  loader.load_from_string(data, target_column)
        │
        ├── Parse reťazca na stĺpce
        ├── Identifikácia target stĺpca
        ├── Konverzia na numerické hodnoty
        │   (ne-numerické → NaN, potom nahradenie priemerom)
        ├── Vytvorenie DenseMatrix<f64> pre X
        └── Extrakcia Vec<f64> pre y
        │
        ▼
  LoadedData { x_data, y_data, headers }
```

Ukážkový dataset

Aplikácia obsahuje predpripravený vzorový dataset s 30 riadkami a 14 stĺpcami pre úlohu schvaľovania úverov:

Stĺpec	Typ	Popis
`age`	Numeric	Vek žiadateľa
`income`	Numeric	Príjem
`credit_score`	Numeric	Kreditné skóre
`loan_amount`	Numeric	Výška pôžičky
`employment_years`	Numeric	Roky zamestnania
`debt_to_income`	Numeric	Pomer dlhu k príjmu
`num_credit_lines`	Discrete	Počet úverových liniek
`late_payments`	Discrete	Počet neskorých platieb
`home_ownership`	Numeric (encoded)	Vlastníctvo nehnuteľnosti
`education_level`	Numeric (encoded)	Úroveň vzdelania
`marital_status`	Numeric (encoded)	Rodinný stav
`has_cosigner`	Binary	Má ručiteľa

Stĺpec	Typ	Popis
`loan_purpose`	Numeric (encoded)	Účel pôžičky
`approved`	**Binary target**	**Schválenie (0/1)**

---

20. Frontendová aplikácia

Technológia

Celá aplikácia je jednostránková (SPA) v jedinom súbore `index.html` (~3600 riadkov): - **CSS**: ~1050 riadkov — vlastný design, responzívny layout - **HTML body**: ~350 riadkov — štruktúra UI - **JavaScript**: ~2200 riadkov — logika aplikácie (ES modules)

Hlavné sekcie UI

```
┌──────────────────────────────────────────────────────────────────────┐
│                      ML Pipeline Application                         │
├──────────────────────────────┬───────────────────────────────────────┤
│                              │                                       │
│   1. VÝBER PIPELINE          │   2. NAČÍTANIE DÁT                    │
│                              │                                       │
│   ├── Model dropdown         │   ├── Formát (CSV/JSON)               │
│   ├── Eval mode              │   ├── Text/File toggle                │
│   ├── Model params           │   ├── Target dropdown                 │
│   └── [Vytvoriť Pipeline]    │   ├── Train/Test split slider         │
│                              │   ├── Target Analysis                 │
│                              │   │   ├── Analyzer cards              │
│                              │   │   └── [Porovnať]                  │
│                              │   └── [Inspect] [Editor]              │
├──────────────────────────────┴───────────────────────────────────────┤
│                                                                      │
│   3. PRIESKUM A VÝBER PRÍZNAKOV (full width)                         │
│                                                                      │
│   ├── Selector comparison grid                                       │
│   │   ├── Variance    ├── Correlation   ├── Chi-Square               │
│   │   ├── Info Gain   ├── MI            ├── SMC                      │
│   │   ├── Synergy VNS ├── Synergy SA                                 │
│   ├── [Porovnať selektory]                                           │
│   ├── Interactive feature map (checkboxes)                           │
│   ├── [Show Heatmap]                                                 │
│   └── Training results comparison                                    │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│   4. DOSTUPNÉ MOŽNOSTI (full width)                                  │
│                                                                      │
│   ├── Models badges    ├── Processors badges                         │
```

```
│    ├── Selectors badges ├── Current pipeline info                │
│    └── Action buttons                                            │
└──────────────────────────────────────────────────────────────────┘
```

Modálne okná

Modal	Funkcia
Heatmap Modal	Fullscreen korelačná/MI matica s tooltipom, sortovaním, legendou
Analyzer Details	Detailná analýza cieľovej premennej
Data Editor	Editácia dát — aplikácia procesorov na stĺpce, mazanie, úprava buniek
Data Inspector	Tabuľkový náhľad na surové/predspracované dáta

JavaScript architektúra

```javascript
// Inicializácia
async function initApp() {
    const wasm = await import('./pkg/wasm.js');
    await wasm.default();           // Init WASM module

    factory = new wasm.WasmFactory();
    pipeline = new wasm.WasmMLPipeline();
    availableOptions = convertWasmResult(factory.getAvailableOptions());

    populateOptions();              // Naplní UI dropdowny
    setupEventListeners();          // Event handlery
}
```

Heatmapa

Interaktívna vizualizácia matice korelácií alebo MI medzi všetkými pármi príznakov:

- **Farebná škála**: Modrá (záporná korelácia) → Biela (0) → Červená (kladná korelácia)
- **MI škála**: Biela (0) → Tmavo červená (vysoká MI)
- **Funkcie**: Prepínanie Correlation/MI, zobrazenie/skrytie hodnôt, zoradenie, tooltip s detailmi
- **Ovládanie**: Escape/tlačidlo na zatvorenie, fullscreen overlay

---

21. Konfigurácia a nasadenie

Požiadavky na build

Nástroj	Verzia	Použitie
Rust	Edition 2021	Jadro aplikácie
wasm-pack	0.14+	Kompilácia do WASM
wasm-bindgen	0.2	JS ↔ Rust bridge

Závislosti (Cargo.toml)

Crate	Verzia	Popis
`smartcore`	0.4	ML algoritmy (regresia, klasifikácia, stromy, KNN)
`kdtree`	0.7	KD-tree pre KSG MI estimátor
`statrs`	0.16	Štatistické funkcie (digamma)
`serde` + `serde_json`	1.0	Serializácia/deserializácia
`csv`	1.2	Parsovanie CSV
`ndarray`	0.15	N-rozmerné polia
`once_cell`	1.19	Lazy static (Singleton)
`getrandom`	0.2 + `js` feature	Generátor náhodných čísel pre WASM
`web-sys`	0.3	Web API prístup (console)

Buildovanie

```
# Kompilácia do WASM
wasm-pack build --target web --out-dir pkg

# Spustenie servera (vyžaduje COOP/COEP hlavičky)
./serve.sh
# alebo
python3 -c "
import http.server, socketserver

class Handler(http.server.SimpleHTTPRequestHandler):
    def end_headers(self):
        self.send_header('Cross-Origin-Opener-Policy', 'same-origin')
        self.send_header('Cross-Origin-Embedder-Policy', 'require-corp')
        super().end_headers()

with socketserver.TCPServer(('', 3333), Handler) as httpd:
    httpd.serve_forever()
"
```

Build a Run skripty

Skript	Popis
`build-and-run.sh`	Skompiluje WASM a spustí server
`serve.sh`	Spustí HTTP server na porte 3333
`run-tests.sh`	Spustí automatizované testy

---

22. Automatizované testovanie — implementácia

Teoretické východiská testovania webových aplikácií a zdôvodnenie výberu Playwright sú opísané v kapitole 7. Testovanie webových aplikácií — Playwright.

Framework

Projekt používa **Playwright** s **pytest** pre E2E (end-to-end) testovanie. Táto kombinácia bola zvolená z nasledovných dôvodov:

- **Playwright** — auto-waiting, podpora WASM asynchrónnosti, `wait_for_function` pre JS conditions
- **pytest** — fixture systém pre setup/teardown, parametrizácia, filtrovanie testov
- **pytest-playwright** — natívna integrácia oboch nástrojov

Nástroj	Verzia
Playwright	1.58+
pytest	7.4+
pytest-playwright	0.7+

Testové súbory a pokrytie

Súbor	Počet testov	Pokrytie	Typ testov
`test_01_init.py`	7	Načítanie stránky, WASM init, zobrazenie modelov/procesorov/selektorov	Smoke testy
`test_02_data_loading.py`	9	Parsovanie CSV, target dropdown, inspect dát, split slider	Funkčné testy
`test_03_pipeline.py`	7	Build pipeline pre každý model, parametre, eval mode	User flow testy

Súbor	Počet testov	Pokrytie	Typ testov
test_04_target_analysis.py	5	Analyzer karty, detaily, porovnanie analyzérov	Integračné testy
test_05_selectors.py	5	Selector karty, select all, porovnanie selektorov	WASM compute testy
test_06_heatmap.py	12	Heatmapa, prepínanie matíc, tooltip, legenda, zoradenie	UI/vizualizačné testy
test_07_editor.py	4	Editor dát, výber stĺpca, procesory	CRUD testy
Celkom	**49**		

Spustenie testov

```
# Všetky testy (headed — s vizuálnym prehliadačom)
python3 -m pytest tests/ --browser chromium -v

# Headless (bez viditeľného prehliadača — pre CI/CD)
python3 -m pytest tests/ --browser chromium --headed false

# Filtrovanie podľa názvu
python3 -m pytest tests/ -k heatmap

# Cez skript
./run-tests.sh
./run-tests.sh --headless
./run-tests.sh -k "pipeline"
```

Testová infraštruktúra

Fixture hierarchy

```
server (session-scoped)        ← 1× za celý test run
  └── app_page (per-test)      ← navigácia + WASM init
        └── loaded_page        ← pipeline + data + target
```

server fixture: - Skontroluje, či server už beží (`socket.connect_ex`) - Ak nie → spustí `serve.sh` ako subprocess s vlastnou process group - Čaká max 15s na dostupnosť portu (polling interval 0.5s) - Po ukončení testov → `SIGTERM` na celú process group (`os.killpg`)

app_page fixture: - Navigácia na `http://localhost:3333` - `wait_until="networkidle"` — počká na načítanie WASM modulu - `wait_for_function` — overí, že WASM inicializácia prebehla (DOM naplnený)

`loaded_page` fixture: - Simuluje kompletný user flow: model → build → CSV → target → confirm - Každý test s `loaded_page` začína s plne funkčným pipeline

`SAMPLE_CSV`: - 30 riadkov, 14 stĺpcov (numerické + binárne) - Binary target "approved" (1/0) — vhodný pre klasifikáciu - Zhodný s ukážkovým datasetom v aplikácii

Postup tvorby testov

Testy boli navrhnuté podľa princípu **pyramídy testovania** zameranej na E2E vrstvu:

1. **Smoke testy** (`test_01`) — overenie, že sa aplikácia vôbec načíta a WASM modul sa inicializuje bez chýb
2. **Data flow testy** (`test_02`) — overenie celého data pipeline od parsovanie CSV po výber targetu
3. **Core functionality** (`test_03-05`) — build pipeline, analýza, selekcia — testovanie jadrovej ML funkcionality
4. **Vizualizácie** (`test_06`) — interaktívna heatmapa, najkomplexnejšia UI komponenta
5. **Editor** (`test_07`) — CRUD operácie nad dátami

Každý test overuje **jednu konkrétnu funkciu** a je pomenovaný podľa konvencie `test_<čo_sa_testuje>`.

---

23. Presety pipeline

Aplikácia obsahuje ~20 predpripravených konfigurácií pipeline:

Klasifikačné presety

Preset	Model	Procesor	Selektor
`basic_classification`	logreg	scaler	variance
`advanced_classification`	logreg	scaler + outlier_clipper	mi
`logreg_minmax_chisquare`	logreg	minmax_scaler	chi_square
`tree_binner_infogain`	tree	binner	information_gain
`knn_classifier`	knn	scaler	variance
`knn_robust_mi`	knn	robust_scaler	mi
`logreg_scaler_only`	logreg	scaler	—
`tree_labelenc_chisquare`	tree	label_encoder	chi_square
`logreg_labelenc_variance`	logreg	label_encoder	variance
`decision_tree`	tree	—	—

Regresné presety

Preset	Model	Procesor	Selektor
`basic_regression`	linreg	scaler	correlation
`advanced_regression`	linreg	scaler + outlier_clipper	mi
`knn_regressor`	knn	scaler	variance
`linreg_minmax_correlation`	linreg	minmax_scaler	correlation
`linreg_robust_mi`	linreg	robust_scaler	mi
`linreg_log_correlation`	linreg	log_transformer	correlation
`knn_power_variance`	knn	power_transformer	variance
`linreg_outlier_correlation`	linreg	outlier_clipper	correlation
`knn_outlier_variance`	knn	outlier_clipper	variance
`linreg_minmax_only`	linreg	minmax_scaler	—
`knn_robust_only`	knn	robust_scaler	—

Špeciálne

Preset	Popis
`minimal`	Žiadny preprocessing ani selekcia

---

24. Používateľská príručka

Krok 1: Vytvorenie Pipeline

1. Vyberte **model** z dropdown menu (logistická regresia, lineárna regresia, KNN, Decision Tree)
2. Nastavte **evaluation mode** (Auto / Classification / Regression)
3. Upravte **parametre modelu** (napr. k pre KNN, max_depth pre Tree)
4. Kliknite **„Vytvoriť Pipeline"**

Krok 2: Načítanie dát

1. Vyberte **formát** (CSV alebo JSON)
2. Vložte dáta do textového poľa alebo nahrajte súbor
   - Môžete použiť **„Stiahnuť ukážkový CSV"** pre rýchly test
3. Kliknite **„Načítať dáta"**
4. Z dropdown menu vyberte **cieľovú premennú** (target)
5. Upravte **pomer train/test** sliderom (predvolené 80/20)
6. Kliknite **„Potvrdiť target"**

Krok 3: Analýza targetu (voliteľné)

1. V sekcii **Target Analysis** vyberte analyzéry (korelácia, MI, SMC, entropia)
2. Kliknite **„Porovnať"** pre porovnanie metód
3. Kliknite na kartu analyzéra pre detailný rozklad

Krok 4: Selekcia príznakov

1. V sekcii **Prieskum a výber príznakov** zaškrtnite požadované selektory
2. Nastavte parametre (počet features, prahy, váhy α/β/γ pre VNS/SA)
3. Kliknite **„Porovnať vybrané selektory”**
4. Prezrite si výsledky — interaktívna mapa príznakov s checkboxami
5. Kliknite **„Heatmap”** pre korelačnú/MI maticu

Krok 5: Tréning a evaluácia

1. Po porovnaní selektorov kliknite **„Trénovať všetky”**
2. Aplikácia natrénuje model s features od každého selektora
3. Porovnajte výsledky — metriky pre každú variantu selekcie

Krok 6: Editor dát (voliteľné)

1. Kliknite **„Editor”** pre otvorenie data editora
2. Vyberte stĺpec a aplikujte processor (encoding, scaling, transformácie)
3. Zmeny sa okamžite premietnu do dát

Dokumentácia generovaná pre ML Pipeline WASM Application v0.1.0